# MULTIVIEW: A Portable C99 System for Two-Camera Scene Analysis and 3D Model Extraction

A self-contained overview of the system and its mathematics

Project documentation — v0.2.0

August 2, 2026

**Abstract**

MULTIVIEW is a small, dependency-free C99 library for metric scene analysis and 3D model extraction from the synchronized video streams of *two stationary cameras* observing an overlapping volume, optionally augmented by a collection of still photographs and, in a later stage, range-finder measurements. Calibration is part of the system: the rig is calibrated from views of a printed letter-page checkerboard of known square size, or from a computer display under our control showing synchronized Gray-code patterns. This document doubles as a self-study overview of the underlying mathematics: each concept is developed from first principles, cross-linked to the modules that implement it, illustrated by a *hand-computable* worked example, and sourced in the bibliography. All quantitative claims (section 12) are reproducible from the deterministic test programs in the repository.

## Contents

# 1 Introduction and capabilities

The target deployment is deliberately narrow and fully under our control: two cameras are rigidly mounted and observe the same working volume. The system first *calibrates itself* from views of a known planar target (section 5); thereafter video is processed frame by frame. Because the cameras do not move, every geometric quantity that depends only on the rig — projection matrices, the fundamental matrix, the rectifying homographies — is computed *once* and reused for every frame, which is the central efficiency win of the stationary-rig assumption.

The implemented core (v0.2.0) provides:

1. plane-based calibration of intrinsics, distortion, and pose from a printed letter-page checkerboard or from a controlled display showing Gray-code patterns (section 5);
2. a calibrated pinhole camera model with Brown radial–tangential lens distortion, forward projection and ray back-projection (section 4);
3. exact two-view epipolar geometry from the calibration, and the normalized 8-point algorithm to estimate or verify it from image correspondences (section 6);
4. $N$-view linear triangulation of corresponding image points into metric 3D, with reprojection-error reporting (section 7);
5. planar stereo rectification of the pair, and dense correspondence by block matching with sub-pixel refinement, giving per-pixel depth (section 8);
6. point-cloud assembly and PLY export for inspection in standard mesh tools, plus minimal PGM image I/O (section 9).

Planned extensions — structure-from-motion over an auxiliary photo collection and fusion of range-finder data — are designed but not yet implemented; their theory and integration points are described in section 10 so that the code can grow into them without rearchitecting.

**How to read this document.** section 3 collects the small amount of linear algebra everything else stands on. section 4–section 8 then follow the data through the system: camera model, calibration, two-view geometry, triangulation, dense stereo. Every section ends in a worked example with numbers chosen so the computation fits on paper; doing them by hand *is* the intended self-study exercise, and each can then be checked against the library.

**Design constraints.** The library is strict C99 with no dependencies beyond `libm`, row-major `double` arrays throughout, and fully deterministic (no clocks, no ambient randomness; the demos use fixed-seed generators). This keeps it portable from desktop to embedded targets, and keeps every experiment repeatable — a prerequisite for the controlled-environment, learning-oriented use the project is intended for.

## 2 System overview

| Module | Header | Responsibility |
|---|---|---|
| `mat` | `mv/mat.h` | small dense linear algebra, Jacobi SVD, null vectors |
| `cam` | `mv/cam.h` | camera model, projection, rays, distortion |
| `target` | `mv/target.h` | printed checkerboard model + printable rendering |
| `graycode` | `mv/graycode.h` | display-based active target (Gray-code patterns) |
| `calib` | `mv/calib.h` | homographies; Zhang calibration; radial estimate |
| `epipolar` | `mv/epipolar.h` | **F** from calibration; normalized 8-point; **E** |
| `triangulate` | `mv/triangulate.h` | $N$-view DLT triangulation, reprojection RMS |
| `rectify` | `mv/rectify.h` | Fusiello rectification, disparity→depth |
| `stereo` | `mv/stereo.h` | dense block matching (SAD, sub-pixel) |
| `img` | `mv/img.h` | PGM (P5) image I/O |
| `cloud` | `mv/cloud.h` | point cloud container, PLY export |

Table 1: Module layout. `mv/mv.h` is the umbrella header.

Processing has a one-time phase and a per-frame phase:

$$\underbrace{\text{target views} \to \text{calibrate}}_{\text{once, } \texttt{target/graycode/calib}} \qquad \underbrace{\text{2 frames} \to \text{undistort} \to \text{rectify} \to \text{match} \to \text{depth} \to \text{cloud}}_{\text{per frame, } \texttt{cam/rectify/stereo/triangulate/cloud}}$$

Sparse operation (feature correspondences instead of dense matching) uses the same chain with `epipolar` providing the correspondence-validation gate. Because the rig is static, temporal accumulation over video frames is a simple union of per-frame clouds in the fixed world frame; moving objects appear as temporally inconsistent points and can be segmented by comparing frames — the entry point for scene *analysis* beyond reconstruction.

## 3 Mathematical toolkit

Four ideas carry the whole system; everything later is an application.

### 3.1 Homogeneous coordinates

A 2D point $(u, v)$ is represented as any triple $(wu, wv, w)$, $w \neq 0$; a 3D point $(X, Y, Z)$ as $(X, Y, Z, 1)$ up to scale. The payoff: projection, homographies and rigid motions all become *linear* maps, and points at infinity ($w = 0$) need no special case. We write $\simeq$ for equality up to a nonzero scale.

**Example 1** (dehomogenization)**.** (120, 140, 2) $\simeq$ (60, 70, 1): both name the pixel $(60, 70)$. Conversely the direction "straight down the $u$ axis" is $(1, 0, 0)$, which no finite pixel represents.

### 3.2 Rotations

A rotation matrix $\mathbf{R} \in SO(3)$ satisfies $\mathbf{R}^\top\mathbf{R} = \mathbf{I}$, $\det \mathbf{R} = 1$; its rows are the axes of the new frame expressed in the old. Consequently $\mathbf{R}^{-1} = \mathbf{R}^\top$ — inverting a rigid transform never requires a matrix inversion.

**Example 2** (rotation about $y$)**.** $\mathbf{R}_y(90^\circ) = \left(\begin{smallmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{smallmatrix}\right)$ sends the world $z$ axis $(0, 0, 1)$ to $(-1, 0, 0)$: a camera yawed 90° sees the world's $-x$ direction ahead. Check $\mathbf{R}^\top\mathbf{R} = \mathbf{I}$ by inspection: the rows are orthonormal.

### 3.3 The cross-product matrix

For $\mathbf{v} = (v_1, v_2, v_3)$,

$$[\mathbf{v}]_\times = \begin{pmatrix} 0 & -v_3 & v_2 \\ v_3 & 0 & -v_1 \\ -v_2 & v_1 & 0 \end{pmatrix}, \qquad [\mathbf{v}]_\times \mathbf{w} = \mathbf{v} \times \mathbf{w}. \tag{1}$$

It is rank 2 with null vector $\mathbf{v}$ itself — the algebraic reason epipoles exist (section 6).

**Example 3** (skew matrix)**.** $\mathbf{v} = (1, 0, 0)$: $[\mathbf{v}]_\times = \left(\begin{smallmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{smallmatrix}\right)$, and $[\mathbf{v}]_\times (0, 1, 0)^\top = (0, 0, 1)^\top = \mathbf{v} \times \mathbf{w}$.

### 3.4 Homogeneous least squares and the SVD

Every estimation problem in this system reduces to: given $\mathbf{A}$ $(m \times n)$, find $\mathbf{x} \neq 0$ minimizing $\|\mathbf{A}\mathbf{x}\|$ subject to $\|\mathbf{x}\| = 1$. The minimizer is the right singular vector of the smallest singular value of $\mathbf{A} = \mathbf{U}\,\mathrm{diag}(\sigma_1 \geq \cdots \geq \sigma_n)\,\mathbf{V}^\top$ [GV13]. The library computes this with a one-sided Jacobi SVD (section 11) via `mv_nullvec`.

**Example 4** (null vector by hand)**.** $\mathbf{A} = \left(\begin{smallmatrix} 1 & 2 \\ 2 & 4 \end{smallmatrix}\right)$ has rank 1 (row 2 = 2× row 1), so the smallest singular value is exactly 0 and the null vector solves $x_1 + 2x_2 = 0$: $\mathbf{x} = \frac{1}{\sqrt{5}}(2, -1)$. Check: $\mathbf{A}\mathbf{x} = \mathbf{0}$.

## 4 Camera model

A camera is a pinhole with intrinsic matrix

$$\mathbf{K} = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}, \tag{2}$$

rotation $\mathbf{R}$ and translation $\mathbf{t}$ mapping world points into the camera frame, $\mathbf{X}_c = \mathbf{R}\mathbf{X} + \mathbf{t}$; the camera center is $\mathbf{C} = -\mathbf{R}^\top \mathbf{t}$. A world point projects to the homogeneous pixel

$$\mathbf{x} \;\simeq\; \mathbf{P}\begin{pmatrix}\mathbf{X}\\ 1\end{pmatrix}, \qquad \mathbf{P} = \mathbf{K}\,[\,\mathbf{R} \mid \mathbf{t}\,] \tag{3}$$

[HZ04]. Real lenses deviate from (3); on normalized coordinates $(x, y) = (X_c/Z_c,\, Y_c/Z_c)$ with $r^2 = x^2 + y^2$ the Brown model gives the distorted

$$\begin{aligned} x_d &= x\,(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2),\\ y_d &= y\,(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy, \end{aligned} \tag{4}$$

and the pixel is $\mathbf{K}(x_d, y_d, 1)^\top$. Back-projection inverts (4) by fixed-point iteration and returns the ray $\mathbf{C} + s\,\mathbf{d}$, $s > 0$.

**Example 5** (projection)**.** Let $f_x{=}f_y{=}100$, $c_x{=}c_y{=}50$, $s{=}0$, $\mathbf{R} = \mathbf{I}$, $\mathbf{t} = \mathbf{0}$, no distortion, and $\mathbf{X} = (1, 2, 10)$. Normalized coordinates: $(x, y) = (1/10,\, 2/10) = (0.1, 0.2)$. Pixel: $u = 100 \cdot 0.1 + 50 = 60$, $v = 100 \cdot 0.2 + 50 = 70$ — the point lands at $(60, 70)$, cf. Example 1.

**Example 6** (camera center)**.** Let $\mathbf{R} = \mathbf{R}_y(90°)$ from Example 2 and $\mathbf{t} = (3, 0, 0)$. Then $\mathbf{R}^\top\mathbf{t} = (0, 0, -3)$ and $\mathbf{C} = -\mathbf{R}^\top\mathbf{t} = (0, 0, 3)$: the camera sits three units down the world $z$ axis. Check forward: $\mathbf{R}\mathbf{C} + \mathbf{t} = (0{\cdot}0 + 0{\cdot}0 - 1{\cdot}3,\; 0,\; 0{\cdot}3) + (3, 0, 0) = (-3, 0, 0) + (3, 0, 0) = \mathbf{0}$ — the center maps to the camera origin, as it must.

# 5 Calibration from planar targets

Calibration is part of the system, not an external input. Both supported targets are *planes with known metric structure*: a printed checkerboard page (subsection 5.4) and a controlled computer display (subsection 5.5). The mathematics is the same for both: several views of a known plane determine the intrinsics, the distortion, and each view's pose, by Zhang's method [Zha00, SM99].

## 5.1 The homography of a plane

Put the target plane at $Z = 0$ in its own frame. Then (3) collapses: writing $\mathbf{R} = [\mathbf{r}_1\,\mathbf{r}_2\,\mathbf{r}_3]$ (columns),

$$\mathbf{x} \simeq \mathbf{K}\,[\mathbf{r}_1\;\mathbf{r}_2\;\mathbf{t}]\,(X, Y, 1)^\top \;=\; \mathbf{H}\,(X, Y, 1)^\top, \tag{5}$$

a $3 \times 3$ *homography* with 8 degrees of freedom. Each point correspondence gives two linear equations in the entries of $\mathbf{H}$, so $n \geq 4$ points determine it as a homogeneous least-squares problem (Example 4 at scale $2n \times 9$), solved by normalized DLT (`mv_homography_dlt`; normalization as in section 11).

**Example 7** (plane homography)**.** Frontal camera: $\mathbf{K}$ as in Example 5, $\mathbf{R} = \mathbf{I}$, $\mathbf{t} = (0, 0, 2)$. Then $[\mathbf{r}_1\,\mathbf{r}_2\,\mathbf{t}] = \left(\begin{smallmatrix}1&0&0\\0&1&0\\0&0&2\end{smallmatrix}\right)$ and

$$\mathbf{H} = \mathbf{K}\begin{pmatrix}1&0&0\\0&1&0\\0&0&2\end{pmatrix} = \begin{pmatrix}100&0&100\\0&100&100\\0&0&2\end{pmatrix} \simeq \begin{pmatrix}50&0&50\\0&50&50\\0&0&1\end{pmatrix}.$$

The plane point $(X, Y) = (0.1, 0.2)$ maps to $(50 \cdot 0.1 + 50,\; 50 \cdot 0.2 + 50) = (55, 60)$. Check via Example 5: the 3D point is $(0.1, 0.2, 2)$, normalized $(0.05, 0.1)$, pixel $(100 \cdot 0.05 + 50,\; 100 \cdot 0.1 + 50) = (55, 60)$.

### 5.2 Zhang's constraints and the closed-form intrinsics

Write $\mathbf{H} = [\mathbf{h}_1\,\mathbf{h}_2\,\mathbf{h}_3]$ (columns). From (5), $\mathbf{h}_1 \simeq \mathbf{K}\mathbf{r}_1$ and $\mathbf{h}_2 \simeq \mathbf{K}\mathbf{r}_2$ with the *same* scale. Since $\mathbf{r}_1, \mathbf{r}_2$ are orthonormal,

$$\mathbf{h}_1^\top \mathbf{B}\,\mathbf{h}_2 = 0, \qquad \mathbf{h}_1^\top \mathbf{B}\,\mathbf{h}_1 = \mathbf{h}_2^\top \mathbf{B}\,\mathbf{h}_2, \qquad \mathbf{B} \;\coloneqq\; \mathbf{K}^{-\top}\mathbf{K}^{-1}, \tag{6}$$

i.e. each view of the plane contributes two linear equations in the six independent entries of the symmetric matrix $\mathbf{B}$ (the *image of the absolute conic*). Three views generically determine $\mathbf{B}$ up to scale; two suffice if zero skew ($s{=}0 \Leftrightarrow B_{12}{=}0$) is imposed. The solution is again a null vector (Example 4); $\mathbf{K}$ then follows from $\mathbf{B}$ in closed form [Zha00], and the pose of each view is recovered as

$$\lambda = \frac{1}{\|\mathbf{K}^{-1}\mathbf{h}_1\|}, \quad \mathbf{r}_1 = \lambda\mathbf{K}^{-1}\mathbf{h}_1, \quad \mathbf{r}_2 = \lambda\mathbf{K}^{-1}\mathbf{h}_2, \quad \mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2, \quad \mathbf{t} = \lambda\mathbf{K}^{-1}\mathbf{h}_3, \tag{7}$$

with $[\mathbf{r}_1\,\mathbf{r}_2\,\mathbf{r}_3]$ snapped to the nearest true rotation via SVD (noise makes it only approximately orthonormal).

**Example 8** (pose from a homography, by hand)**.** Take the (unscaled) $\mathbf{H}$ of Example 7 and the known $\mathbf{K}$. Then

$$\mathbf{K}^{-1} = \begin{pmatrix} 0.01 & 0 & -0.5 \\ 0 & 0.01 & -0.5 \\ 0 & 0 & 1 \end{pmatrix}, \quad \mathbf{K}^{-1}\mathbf{h}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \mathbf{K}^{-1}\mathbf{h}_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \quad \mathbf{K}^{-1}\mathbf{h}_3 = \begin{pmatrix} 0 \\ 0 \\ 2 \end{pmatrix}.$$

Both constraint equations of (6) hold: $\mathbf{r}_1{\cdot}\mathbf{r}_2 = 0$ and $\|\mathbf{r}_1\| = \|\mathbf{r}_2\|$. The scale is $\lambda = 1/\|(1,0,0)\| = 1$, so (7) gives $\mathbf{r}_1 = (1,0,0)$, $\mathbf{r}_2 = (0,1,0)$, $\mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2 = (0,0,1)$, $\mathbf{t} = (0,0,2)$ — exactly the frontal pose two metres from the target that we started from.

### 5.3 Radial distortion and alternation

With $\mathbf{K}$ and poses in hand, the ideal (undistorted) projection $(u, v)$ of every target point is known, and (4) predicts the observed $(u_d, v_d)$ linearly in $(k_1, k_2)$:

$$u_d - u = (u - c_x)(k_1 r^2 + k_2 r^4), \qquad v_d - v = (v - c_y)(k_1 r^2 + k_2 r^4), \tag{8}$$

a least-squares problem with two unknowns. Since the homographies were fit to *distorted* points, the library alternates: fit geometry, estimate $(k_1, k_2)$, undistort the observations, refit — a few rounds converge for moderate distortion [Zha00].

An identifiability caveat worth internalizing: over the small radius range a letter-page target covers, $r^2$ and $r^4$ are nearly proportional, so $k_1$ and $k_2$ are individually ill-determined even when their combined *curve* $k_1 r^2{+}k_2 r^4$ — the thing that actually corrects pixels — is fit to well below noise (measured in section 12). Wide targets, or the full-field display target of subsection 5.5, are the cure when the individual coefficients matter.

### 5.4 The printed target

The standard passive target is a checkerboard printed on a letter/A4 page: $8 \times 10$ squares of nominally 24 mm ($192 \times 240$ mm), i.e. $7 \times 9$ inner corners (`mv/target.h`; `mv_target_render_pgm` emits the printable image — at 10 px/mm a 24 mm square is 240 px). Printers rescale, so the

procedure is *print, then measure one square* and pass the measured size; only that measurement makes the calibration metric. Corner *positions* in the image are currently supplied by the caller (an external detector or manual marking); a saddle-point corner detector is on the roadmap (section 13). Ten to twenty views with strong tilt diversity (±30–40°) condition the focal length well — near-frontal views leave it weakly observable, a property visible in the experiment of section 12.

### 5.5 The display as an active target

A computer display we control is a better target than paper wherever one is available: its pixel grid is a plane of exactly known geometry (pixel pitch from the panel datasheet, or measured once), and because we control what it shows *when*, correspondence needs no detector at all. The display plays, synchronized with capture, a sequence of binary stripe patterns that encode each display pixel's coordinate in *Gray code* [ISM84, Gen11]: bit $b$ of the code is shown as stripes, followed by its inverse, for $b = 0..\lceil \log_2 W \rceil - 1$, then the same for the $y$ axis. A camera pixel observing the display reads its bit as "brighter in the pattern or in the inverse?" — a per-pixel comparison immune to illumination, display brightness and gamma — and after all frames has decoded *which display pixel it sees* (`mv/graycode.h`). This yields thousands of dense, uniformly spread correspondences per view; the same pipeline (subsection 5.1–subsection 5.3) consumes them, with plane coordinates = display pixel × pitch. Gray code rather than plain binary because adjacent stripes differ in exactly one bit: a threshold error at a stripe boundary displaces the decoded coordinate by at most one pixel instead of, e.g., $2^b$.

**Example 9** (Gray code by hand)**.** Encode $v = 6$: binary 110; Gray $= 110 \oplus 011 = 101$. Decode $g = 101$ by prefix-XOR: $b_2 = 1$, $b_1 = 1 \oplus 0 = 1$, $b_0 = 1 \oplus 0 \oplus 1 = 0$ — binary $110 = 6$. Now flip the last Gray bit (a boundary misread): $g' = 100 \to$ binary $111 = 7$, an error of exactly one position. In plain binary, flipping the top bit of 110 gives $010 = 2$, an error of four.

## 6 Two-view epipolar geometry

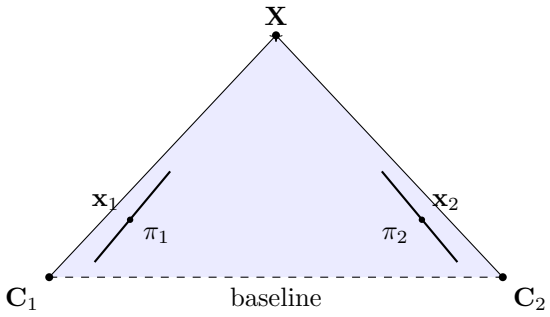


Figure 1: Epipolar geometry: the camera centers $\mathbf{C}_1, \mathbf{C}_2$ and the scene point $\mathbf{X}$ span the epipolar plane (shaded); its intersections with the image planes $\pi_1, \pi_2$ are the corresponding epipolar lines on which $\mathbf{x}_1$ and $\mathbf{x}_2$ must lie.

Corresponding pixels of the two views are linked by the fundamental matrix $\mathbf{F}$ through the epipolar constraint

$$\mathbf{x}_2^\top \mathbf{F}\, \mathbf{x}_1 = 0, \tag{9}$$

with $\mathbf{F}$ of rank 2; the line $\boldsymbol{\ell}_2 = \mathbf{F}\mathbf{x}_1$ is the locus in image 2 of possible matches of $\mathbf{x}_1$ (Figure 1). For *calibrated* views the essential matrix $\mathbf{E} = \mathbf{K}_2^\top \mathbf{F}\, \mathbf{K}_1 = [\mathbf{t}]_\times \mathbf{R}$ carries the same constraint in normalized coordinates [LH81]; note the reappearance of (1).

Because our rig is calibrated, $\mathbf{F}$ is available in closed form,

$$\mathbf{F} = [\, e_2 \,]_\times \, \mathbf{P}_2 \, \mathbf{P}_1^{+}, \qquad e_2 = \mathbf{P}_2 \begin{pmatrix} \mathbf{C}_1 \\ 1 \end{pmatrix}, \tag{10}$$

where $\mathbf{P}_1^{+}$ is the pseudo-inverse of $\mathbf{P}_1$ [HZ04]. The system computes (10) once at start-up and uses it for *correspondence gating*: any candidate match whose symmetric epipolar distance exceeds a threshold is rejected before triangulation.

Independently, $\mathbf{F}$ can be *estimated* from $n \geq 8$ correspondences by the normalized 8-point algorithm [Har97]: each pair turns (9) into one linear equation in the nine entries of $\mathbf{F}$ — again the homogeneous least-squares pattern of Example 4 — followed by rank-2 enforcement via SVD. In this system the estimator serves two purposes: (i) *health monitoring* — if the estimate from live correspondences drifts from the calibrated (10), the rig has been bumped or the calibration is stale; (ii) bootstrapping geometry for *uncalibrated* auxiliary photos (section 10). On contaminated correspondence sets the estimator is wrapped in a RANSAC loop [FB81] (planned with the feature layer).

**Example 10** (essential matrix of a rectified rig)**.** Two identical cameras, the second displaced one unit along $x$: $\mathbf{R} = \mathbf{I}$, $\mathbf{C}_2 = (1, 0, 0)$, so $\mathbf{t} = -\mathbf{R}\mathbf{C}_2 = (-1, 0, 0)$. By Example 3,

$$\mathbf{E} = [\mathbf{t}]_\times \mathbf{R} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{pmatrix}, \qquad \mathbf{x}_2^\top \mathbf{E}\, \mathbf{x}_1 = (x_2, y_2, 1) \begin{pmatrix} 0 \\ 1 \\ -y_1 \end{pmatrix} = y_2 - y_1.$$

The epipolar constraint degenerates to $y_2 = y_1$: corresponding points lie on the *same image row*. This is precisely the situation rectification (section 8) manufactures for arbitrary rigs.

## 7 Triangulation

Given observations $\mathbf{x}^{(v)} = (u_v, w_v)$ of the same point in $N \geq 2$ views with projection matrices $\mathbf{P}^{(v)}$, each view contributes two linear equations obtained by eliminating the projective scale from (3):

$$\begin{pmatrix} u_v\, \mathbf{P}^{(v)}_{3,:} - \mathbf{P}^{(v)}_{1,:} \\ w_v\, \mathbf{P}^{(v)}_{3,:} - \mathbf{P}^{(v)}_{2,:} \end{pmatrix} \begin{pmatrix} \mathbf{X} \\ 1 \end{pmatrix} = \mathbf{0}, \tag{11}$$

and stacking all views gives a $2N \times 4$ homogeneous system solved, once more, as in Example 4 — the DLT triangulation [HZ04]. The implementation accepts any $N \geq 2$, so still photographs registered into the rig frame (section 10) simply extend the same call. DLT minimizes an algebraic residual; the reprojection-optimal two-view method of Hartley–Sturm [HS97] is a planned refinement, worthwhile mainly for wide-baseline auxiliary views. The library reports the RMS reprojection error of every triangulated point — the quantity an eventual bundle adjustment would minimize [TMHF00].

**Example 11** (triangulation by similar triangles)**.** Rig of Example 10 with $\mathbf{K}$ of Example 5 ($f = 100$, $c = 50$) and baseline $B = 0.2$: camera 2 at $(0.2, 0, 0)$. Scene point $\mathbf{X} = (0.1, 0, 2)$.

View 1: $(x, y) = (0.05, 0)$, pixel $u_1 = 100 \cdot 0.05 + 50 = 55$. View 2: camera coordinates $(0.1 - 0.2,\ 0,\ 2) = (-0.1, 0, 2)$, so $u_2 = 100 \cdot (-0.05) + 50 = 45$. Disparity $d = u_1 - u_2 = 10$, and (12) gives $Z = fB/d = 100 \cdot 0.2/10 = 2$ ✓. The lateral coordinate follows from view 1: $X = Z\,(u_1 - c_x)/f = 2 \cdot 5/100 = 0.1$ ✓— the full point recovered by hand.

## 8 Rectification and dense stereo

For dense per-pixel depth the pair is first *rectified*: both cameras are rotated (synthetically, about their centers) onto a common orientation whose $x$ axis is the baseline, making the geometry of Example 10 true by construction — epipolar lines become identical image rows and correspondence search collapses to 1D. We use the compact algorithm of Fusiello, Trucco and Verri [FTV00]: the new rotation has rows $(\mathbf{x}, \mathbf{y}, \mathbf{z})$ with $\mathbf{x}$ the unit baseline, $\mathbf{y} = \mathbf{k} \times \mathbf{x}$ ($\mathbf{k}$ the old optical axis of camera 1) and $\mathbf{z} = \mathbf{x} \times \mathbf{y}$; both views share the averaged intrinsics with zero skew, and the pixel maps are the homographies $\mathbf{H}_i = \mathbf{K}_{\text{new}}\mathbf{R}_{\text{new}}\mathbf{R}_i^{\top}\mathbf{K}_i^{-1}$ (homographies again — cf. (5), here induced by pure rotation instead of a scene plane). Since the rig is stationary, $\mathbf{H}_1, \mathbf{H}_2$ are computed once and each video frame is warped by table lookup.

On the rectified pair, dense correspondence is found by block matching: for each left pixel the sum of absolute differences (SAD) over a $(2w{+}1)^2$ window is minimized along the corresponding right row over a disparity range, followed by sub-pixel refinement by parabolic interpolation of the cost around the minimum. This is the classic local baseline method in the taxonomy of Scharstein and Szeliski [SS02] — chosen deliberately: it is simple, portable, trivially parallel, and its failure modes (textureless regions, occlusions, repetitive patterns) are well understood, giving a controlled baseline against which future matchers (rank/census transforms, semi-global aggregation) can be measured within the same harness.

Matched disparity $d$ converts to metric depth by

$$Z = \frac{f\,B}{d}, \tag{12}$$

with $f$ the shared rectified focal length (pixels) and $B$ the baseline. Differentiating (12) gives the expected depth uncertainty

$$\sigma_Z \;=\; \frac{Z^2}{f\,B}\,\sigma_d \tag{13}$$

for disparity noise $\sigma_d$ — the quadratic growth with $Z$ is the fundamental accuracy law of the rig.

**Example 12** (depth error budget)**.** The experimental rig of section 12: $f = 800\,\text{px}$, $B = 0.5\,\text{m}$, mean depth $Z = 4.7\,\text{m}$, per-coordinate noise $\sigma = 0.3\,\text{px}$ so $\sigma_d = \sqrt{2}\,\sigma \approx 0.42\,\text{px}$. Then $\sigma_Z = 4.7^2 \cdot 0.42/(800 \cdot 0.5) \approx 0.023\,\text{m}$: about 2.3 cm of depth uncertainty — compare the measured 2.65 cm RMS 3D error in Table 2.

## 9 From depth to models

Every depth estimate back-projects to a 3D point in the fixed world frame; points are accumulated in a growable cloud with optional per-point color and exported as ASCII PLY, directly loadable in MeshLab, CloudCompare or Blender for inspection. Over a video sequence the static-rig assumption makes accumulation a union in a common frame; per-point temporal statistics

(persistence, variance) separate static structure from moving objects. The planned meshing path is classical: volumetric range-image fusion [CL96] or Poisson surface reconstruction [KBH06] over the accumulated cloud; both consume exactly the oriented-point data the pipeline produces, so they attach cleanly as later modules.

# 10 Planned extensions

## 10.1 Auxiliary photo collections

Still photographs of the same volume — arbitrary viewpoints, possibly a different camera — densify coverage and fill occlusions. The integration path follows the structure-from-motion recipe of Photo Tourism [SSS06]: detect scale-invariant features [Low04] in the photos *and* in the two calibrated video frames; match with a RANSAC gate [FB81]; register each photo to the rig's world frame by resection from matches against already-triangulated points; then re-triangulate all tracks with the registered views via the existing $N$-view DLT (section 7); finally refine everything with bundle adjustment [TMHF00]. The two calibrated cameras anchor the metric scale — the classic scale ambiguity of pure SfM does not arise. Dense multiview stereo over the registered set (e.g. patch-based methods [FP10]) is a further optional stage.

## 10.2 Range-finder fusion

A range finder (laser/ToF) contributes sparse but metrically strong depth. Two integration modes are planned: (i) if the range finder is rigidly mounted and extrinsically calibrated to the rig, each range sample is simply another world-frame point with small $\sigma_Z$, fused into the cloud with inverse-variance weights; (ii) if it produces an independent scan, the scan is registered to the vision cloud by closed-form absolute orientation [Hor87] initialized correspondence-free and refined with ICP [BM92]. In both modes the range data additionally serves as a ground-truth audit of the stereo depth law (13).

# 11 Numerical implementation notes

All decompositions reduce to one kernel: a one-sided (Hestenes) Jacobi SVD [GV13] for $m \geq n$ matrices, $n \leq 9$ in every present use (the design matrices of (5), (6), (11), and the 8-point system). Jacobi SVD was chosen over Golub–Kahan bidiagonalization for its simplicity (∼100 lines, no workspace queries), unconditional convergence, and excellent relative accuracy on small matrices; its $O(mn^2)$-per-sweep cost is irrelevant at these sizes. Homogeneous solutions are the right singular vector of the smallest singular value; rank-2 enforcement for $\mathbf{F}$ zeroes the smallest singular value and recomposes.

Conditioning is handled where it matters: design matrices built from raw pixel coordinates mix terms of order 1, $u$, and $u^2$ ($u \sim 10^2$ pixels), giving condition numbers around $10^8$; Hartley normalization [Har97] (translate to centroid, scale to RMS radius $\sqrt{2}$) reduces this to $O(10)$ and is applied in both the 8-point and the homography estimators, with results denormalized afterwards. All fundamental matrices are canonicalized (unit Frobenius norm, largest-magnitude entry positive) so that analytic and estimated matrices are directly comparable — this is what makes the drift monitor of section 6 a plain matrix norm. Calibration runs Zhang's closed form inside the small alternation loop of subsection 5.3; a full Levenberg–Marquardt refinement over all parameters is the planned upgrade (section 13).

The code is strict C99 (`-std=c99 -pedantic -Wall -Wextra` clean), depends only on `libm`, allocates only in unavoidable places (design matrices, images, clouds), and contains no hidden state except the demos' seeded generators. Determinism is treated as a feature: identical inputs give bit-identical outputs across runs, so every regression is reproducible.

## 12 Basic results

Three validation layers accompany the library. `tests/test_mv.c` checks exact properties on noiseless data (30 assertions: SVD reconstruction and orthonormality, ray/projection consistency, (9) to $10^{-8}$, exact homography and triangulation recovery, exact intrinsic/pose recovery by Zhang's method, Gray-code encode/decode round-trip, row alignment after rectification); all pass. The two demo programs are controlled noisy experiments.

**Two-camera reconstruction (`demo/synthetic.c`).** Two cameras with $f = 800$ px, $640\times480$ images, baseline $B = 0.5$ m, the second camera yawed 6° inward, observe 250 points drawn uniformly in a $2 \times 1.5$ m window at depths 3.5–6 m; projections carry Gaussian noise $\sigma = 0.3$ px per coordinate (seed fixed).

| Quantity | Value |
|---|---|
| points visible in both views | 250/250 |
| $\|\mathbf{F}_{\text{analytic}} - \mathbf{F}_{\text{8-point}}\|_F$ (unit-norm) | $4.5 \times 10^{-3}$ |
| mean symmetric epipolar distance (analytic $\mathbf{F}$) | 0.345 px |
| mean symmetric epipolar distance (8-point $\mathbf{F}$) | 0.341 px |
| triangulation RMS 3D error | 0.0265 m |
| triangulation max 3D error | 0.0876 m |
| RMS reprojection error | 0.218 px |
| mean row misalignment $\|v_1 - v_2\|$ before rectification | 0.764 px |
| mean row misalignment after rectification | 0.0 px |

Table 2: Reconstruction experiment, seed 42 (reproduce with `make demo`).

The numbers agree with theory: the epipolar residual $\approx 0.34$ px matches the injected noise, and the RMS 3D error of 2.65 cm matches the 2.3 cm depth-noise floor computed by hand in Example 12 (the measured value also contains the smaller lateral components). The implementation performs at the noise floor of the geometry rather than being implementation-limited. The reconstructed cloud is written to `out_cloud.ply`.

**Calibration (`demo/calibrate.c`).** A camera with ground-truth $f_x$=800, $f_y$=805, $(c_x, c_y)$=(322, 238), zero skew and zero distortion photographs the letter-page target (subsection 5.4; $7 \times 9$ corners, 24 mm squares) in six poses with tilts up to 35° at 0.45–0.65 m; corners carry $\sigma = 0.3$ px noise.

Sub-1% intrinsics and millimetre-level poses from six noisy views of one printed page is the expected performance of Zhang's method without nonlinear refinement; near-frontal pose sets degrade the focal estimate markedly (weak observability, subsection 5.4), which the experiment reproduces if the tilt angles are reduced. The fitted $(k_1, k_2)$ absorb a little noise as a near-zero distortion *curve* while being individually large — the identifiability effect discussed in

| Quantity | Value |
|---|---|
| $f_x$ error | $-7.0$ px (0.9%) |
| $f_y$ error | $-7.9$ px (1.0%) |
| $c_x$, $c_y$ error | $-1.0$, $-1.8$ px |
| mean rotation error over views | $0.31^\circ$ |
| mean translation error over views | 5.2 mm |
| reprojection RMS | 0.48 px |

Table 3: Calibration experiment, seed 7, six views, $\sigma = 0.3$ px (reproduce with `make demo`).

subsection 5.3. A Levenberg–Marquardt refinement stage is the roadmap item that tightens all of these numbers.

## 13 Roadmap

In dependency order: (1) saddle-point checkerboard corner detection with sub-pixel refinement, closing the printed-target path end-to-end (the display path of subsection 5.5 is already detector-free); (2) Levenberg–Marquardt refinement of calibration (and later bundle adjustment [TMHF00]); (3) feature detection/description and robust matching (enables live correspondences, RANSAC-gated by section 6); (4) image warping by the rectifying homographies with bilinear resampling, closing the dense video path end-to-end; (5) temporal cloud statistics for static/moving segmentation (section 9); (6) photo-collection registration (subsection 10.1); (7) Hartley–Sturm optimal triangulation; (8) range-finder fusion (subsection 10.2); (9) meshing. Each step lands as a new module behind the same conventions (Table 1) with its own noiseless-exact tests and a synthetic experiment extending section 12.

## References


[BM92] Paul J. Besl and Neil D. McKay. A method for registration of 3-d shapes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 14(2):239–256, 1992.

[CL96] Brian Curless and Marc Levoy. A volumetric method for building complex models from range images. In *Proceedings of SIGGRAPH 96*, pages 303–312, 1996.

[FB81] Martin A. Fischler and Robert C. Bolles. Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.

[FP10] Yasutaka Furukawa and Jean Ponce. Accurate, dense, and robust multiview stereopsis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(8):1362–1376, 2010.

[FTV00] Andrea Fusiello, Emanuele Trucco, and Alessandro Verri. A compact algorithm for rectification of stereo pairs. *Machine Vision and Applications*, 12(1):16–22, 2000.

[Gen11] Jason Geng. Structured-light 3d surface imaging: a tutorial. *Advances in Optics and Photonics*, 3(2):128–160, 2011.

[GV13] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, fourth edition, 2013.

[Har97] Richard I. Hartley. In defense of the eight-point algorithm. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 19(6):580–593, 1997.

[Hor87] Berthold K. P. Horn. Closed-form solution of absolute orientation using unit quaternions. *Journal of the Optical Society of America A*, 4(4):629–642, 1987.

[HS97] Richard I. Hartley and Peter Sturm. Triangulation. *Computer Vision and Image Understanding*, 68(2):146–157, 1997.

[HZ04] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, second edition, 2004.

[ISM84] Seiji Inokuchi, Kosuke Sato, and Fumio Matsuda. Range imaging system for 3-d object recognition. In *Proceedings of the International Conference on Pattern Recognition*, pages 806–808, 1984.

[KBH06] Michael Kazhdan, Matthew Bolitho, and Hugues Hoppe. Poisson surface reconstruction. In *Proceedings of the Fourth Eurographics Symposium on Geometry Processing*, pages 61–70, 2006.

[LH81] H. C. Longuet-Higgins. A computer algorithm for reconstructing a scene from two projections. *Nature*, 293:133–135, 1981.

[Low04] David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004.

[SM99] Peter F. Sturm and Stephen J. Maybank. On plane-based camera calibration: A general algorithm, singularities, applications. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 432–437, 1999.

[SS02] Daniel Scharstein and Richard Szeliski. A taxonomy and evaluation of dense two-frame stereo correspondence algorithms. *International Journal of Computer Vision*, 47(1–3):7–42, 2002.

[SSS06] Noah Snavely, Steven M. Seitz, and Richard Szeliski. Photo tourism: Exploring photo collections in 3d. *ACM Transactions on Graphics*, 25(3):835–846, 2006.

[TMHF00] Bill Triggs, Philip F. McLauchlan, Richard I. Hartley, and Andrew W. Fitzgibbon. Bundle adjustment — a modern synthesis. In *Vision Algorithms: Theory and Practice*, volume 1883 of *Lecture Notes in Computer Science*, pages 298–372. Springer, 2000.

[Zha00] Zhengyou Zhang. A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11):1330–1334, 2000.